CA

CareAxis
Clinic Appointment System

COURSE PROJECT REPORT

Clinic Appointment System Using MERN Stack

A visually designed academic report and case study for a zero-cost healthcare management web application built with React, Node.js, Express, and MongoDB.

3 roles supported

10+ major modules and flows

0 paid services required

SUBMISSION DETAILS

Project Title	CareAxis Clinic Appointment System
Domain	Medical / Healthcare Management
Technology	MERN Stack
Type	Zero-cost Full Stack Web Application
Date	14 April 2026
Prepared For	Activity Based / Practical Based Learning

PROJECT FOCUS

The application digitizes clinic scheduling by giving patients, doctors, and administrators role-specific tools for appointments, availability, and reporting.

Designed submission document

CareAxis Project Report

OVERVIEW

Abstract and Table of Contents

01

Abstract

CareAxis is a full-stack clinic appointment platform created to replace manual booking registers and fragmented scheduling practices with a centralized digital system. The application uses React for the user interface, Express and Node.js for backend APIs, and MongoDB for persistent storage. It includes JWT-based authentication, role-based permissions, doctor profile management, appointment validation, dashboards, and case-oriented workflow support.

The project is fully suitable for academic use because it demonstrates practical MERN concepts without relying on paid APIs, payment gateways, cloud storage, or third-party communication services.

Contents

1. Introduction and Problem Context 3

2. Objectives and Scope 3

3. Existing vs Proposed System 4

4. Architecture and Technology Stack 4

5. Modules and Role Permissions 5

6. Database Design and API Summary 6

7. Case Study and Workflow Analysis 7

8. Testing, Results, and Future Scope 8

9. Conclusion 8

JWT
secure authentication

CRUD
full record management

Charts
dashboard analytics

MongoDB
local zero-cost database

Abstract and project map Page 2

FOUNDATION

Introduction, Problem Statement, and Objectives

02

Introduction

Small clinics often depend on notebooks, calls, spreadsheets, and informal staff coordination to manage appointments. As patient volume grows, this manual workflow creates delays, confusion, and avoidable booking conflicts. A digital appointment platform can improve the quality of clinic operations while also reducing the administrative burden on staff.

CareAxis addresses this challenge by providing a centralized web application for administrators, doctors, and patients. Each role sees only the features required for its tasks, making the system simple, secure, and efficient.

Problem Statement

- Manual appointment handling causes booking errors and double scheduling.
- Doctor availability is difficult to verify in real time.
- Patients do not have a clear self-service booking workflow.
- Clinic staff lack a single dashboard for monitoring appointment status.
- There is no structured role-based system for tracking consultation progress.

Main Objective

Build a practical MERN application that digitizes clinic scheduling and workflow management.

Academic Value

Demonstrate authentication, CRUD, dashboards, validation, schema design, and role-based routing.

Zero-Cost Constraint

Run locally with MongoDB Community Server and avoid paid APIs or cloud dependencies.

Specific Objectives

- Provide secure login for admin, doctor, and patient roles.
- Allow admin users to create and manage doctor accounts.
- Prevent duplicate bookings for the same doctor and slot.
- Allow doctors to confirm, complete, or cancel appointments.

- Enable patients to browse specialists and request appointments.

- Present dashboard summaries for each role.

DESIGN

Scope, Existing System, Proposed System, and Architecture

03

Existing Manual System

- Appointments are written in registers or managed over phone calls.
- Doctor schedules are coordinated manually by staff.
- Appointment status tracking is inconsistent.
- Reports are difficult to prepare and update.

Proposed Digital System

- Centralized appointment storage in MongoDB.
- Doctor directory with specialization and availability details.
- Role-based dashboards for admin, doctor, and patient.
- Booking validation to block unavailable or duplicate slots.

SYSTEM ARCHITECTURE

Three-Layer MERN Application Flow

React Frontend

Public pages, protected dashboards, forms, doctor listing, appointment views, and charts.

Express REST API

Authentication, doctor management, appointment validation, dashboard aggregation, and access control.

MongoDB Database

Stores users, doctor profiles, appointment records, status history, and timestamps.

Included in Scope

Registration, login, role control, appointment booking, doctor

Excluded from Scope

Online payment, SMS reminders, telemedicine, prescription PDFs, and cloud deployment.

Why This Scope Works

It is realistic, manageable for academic evaluation, and rich enough to show full-stack capability.

management, and analytics dashboards.

IMPLEMENTATION

Technology Stack, Modules, and User Roles

04

Frontend

- React.js
- Vite
- React Router DOM
- Recharts
- Custom CSS UI

Backend

- Node.js
- Express.js
- JWT Authentication
- bcryptjs
- Morgan and CORS

Database

- MongoDB Community Server
- Mongoose Models
- Schema validation
- Unique appointment indexing

Main Application Modules

Authentication Module

Register patient accounts, login users, issue JWT tokens, and load current user context.

Doctor Management Module

Create doctor accounts, store specializations, fees, bios, and weekly availability.

Appointment Module

Book, list, update, and cancel appointments with role-wise access rules.

Dashboard Module

Summaries, trends, top doctors, care team visibility, and workload analysis.

User Roles and Permissions

ADMIN

- Create doctor accounts
- View all appointments
- Update doctor details

DOCTOR

- View assigned appointment queue
- Add doctor notes
- Confirm or complete visits

PATIENT

- Register and log in
- Browse doctor directory
- Book or cancel appointments
- Track appointment history

- Track dashboard metrics

- Edit own profile details

DATA AND APIS

Database Design and REST Endpoint Summary

05

MongoDB Collection Design

Collection	Important Fields	Purpose
User	name, email, password, role, phone, gender, age, lastLoginAt	Stores admin, doctor, and patient login identity.
DoctorProfile	user, specialization, qualifications, experienceYears, consultationFee, location, availability	Stores doctor-specific metadata and schedule slots.
Appointment	patient, doctor, appointmentDate, reason, status, doctorNotes, cancellationReason	Tracks the lifecycle of each booked consultation.

A unique compound index on doctor and appointmentDate prevents duplicate bookings for the same slot.

Authentication APIs

Method	Endpoint
POST	/api/auth/register
POST	/api/auth/login
GET	/api/auth/me

Doctor APIs

Method	Endpoint
GET	/api/doctors
GET	/api/doctors/:id
POST	/api/doctors
PUT	/api/doctors/:id

Appointment APIs

Method	Endpoint
GET	/api/appointments
POST	/api/appointments
PATCH	/api/appointments/:id/status

Dashboard API

Method	Endpoint
GET	/api/dashboard/overview

Returns role-specific summary metrics and chart-friendly aggregates.

PATCH	/api/appointments/:id/cancel

CASE STUDY

Workflow Analysis and Real-World Scenario

06

Case Background

Consider a small neighborhood clinic with one administrator, several doctors, and a growing number of patients. Previously, the clinic managed appointments using calls, handwritten notes, and informal staff coordination. This resulted in delays, missed updates, and difficulty tracking the status of consultations.

1 Admin Setup

The administrator creates doctor accounts with specialization, fee, and weekly availability.

2 Patient Registration

The patient creates an account and opens the doctor directory.

3 Appointment Request

The patient selects a doctor, a valid future date and time, and enters the reason for consultation.

4 Validation

The system verifies doctor existence, future date, availability window, and slot uniqueness.

5 Doctor Action

The doctor reviews the queue, adds notes, and updates status to confirmed, completed, or cancelled.

6 Admin Monitoring

The administrator tracks totals, trends, recent bookings, and doctor performance.

Illustrative Scenario

Riya Sharma logs in as a patient and chooses Dr. Asha Menon for recurring headache and fever. She selects a future slot inside the doctor's published schedule. The application accepts the request only if the slot is free. Dr. Asha later confirms the booking and completes the consultation after the visit. The updated status then appears in the patient dashboard.

Case Outcome

- No manual phone coordination is required for basic booking.
- The clinic avoids duplicate scheduling.
- Doctors get a clear queue view.
- Patients can see appointment progress transparently.

EVALUATION

Testing, Results, Future Enhancements, and Conclusion

07

Technical Checks

- Client linting passed
- Production build passed
- Server syntax checks passed

Functional Outcomes

- Admin can create doctors
- Patients can request appointments
- Doctors can update status and notes

Business Impact

- Faster booking workflow
- Reduced manual coordination
- Improved clinic visibility

Future Enhancements

- Email and SMS appointment reminders
- Prescription and report management
- PDF export and advanced reporting
- Multi-branch clinic support
- Telemedicine and video consultation

Conclusion

CareAxis demonstrates how a small healthcare workflow can be transformed using a well-structured MERN application. The system combines authentication, validation, dashboards, database design, and role-based interaction into a project that is both academically strong and practically meaningful.

Because it is zero-cost, realistic, and technically complete, it is well suited for a practical course project in web development or full stack development.

References

React Documentation, Express.js Documentation, MongoDB Documentation, Node.js Documentation, JSON Web Token concepts, and practical MERN stack development references.